

AdaHome: 一种使用本地小语言模型的自适应智能家居助手*

Eu Jin Lim 南安普顿大学
南安普顿, 英国
ejlle22@soton.ac.uk

Zhaoxing Li 南安普顿大学
南安普顿, 英国
zhaoxing.li@soton.ac.uk

Sebastian Stein 南安普顿大学
南安普顿, 英国
ss2@ecs.soton.ac.uk

摘要

智能家居助手能够解读广泛的用户指令, 从明确的设备控制到定义不明确且依赖于偏好的请求。虽然近期基于大语言模型的系统提升了此能力, 但它们通常依赖于重量级的推理流水线和基于云的部署, 限制了其在资源受限环境下的效率和适用性, 并引发了隐私担忧。此外, 现有方法对稳定的长期个性化支持有限。为解决这些问题, 我们提出了AdaHome, 一种专为智能家居环境中本地部署的小语言模型设计的自适应智能家居助手。AdaHome并未统一应用复杂的推理, 而是引入了一个意图感知的规划框架, 动态地将指令路由到基于简单提示的组件或基于轻量级推理的组件。对于需要解读的指令, 我们采用链式草稿策略来实现高效且稳定的决策。为支持个性化, 我们进一步提出了一种偏好适应机制, 该机制能够从长期的用户反馈中学习, 而无需提示增强或模型重训练。我们在统一小模型设置下, 将AdaHome与代表性的大语言模型基线进行了评估。AdaHome在直接指令上实现了显著更高的准确率 (86.7%), 同时将延迟降低了高达 3x。此外, 它在模糊输入上保持了有竞争力的性能, 且计算成本更低。在多轮场景中, AdaHome实现了88%的偏好一致性, 而提示增强基线仅为52.5%。

CCS概念

• 以人为中心的计算 → 交互设备; • 计算方法论 → 自然语言处理。

关键词

智能家居, 小语言模型, 自适应推理, 个性化, 偏好适应

1 引言

使用自然语言与智能家居环境交互仍然具有挑战性。用户通常以目标、情境或偏好来表达指令, 例如“让房间舒适”或“我想放松一下” [7]。然而, 智能家居系统需要明确的指令来指定要控制哪些设备以及执行何种

操作。这在用户的表达方式与系统的指令执行方式之间造成了根本性的鸿沟。弥合这一鸿沟需要从那些未充分指定且模糊的输入中推断出具体操作。大语言模型的集成提升了这方面的能力 [2, 6]。大语言模型能够基于上下文信息进行推理, 并生成结构化的行动计划, 从而支持超越传统基于规则流程的更灵活交互 [7, 14, 16]。

尽管取得了这些进展, 实际部署仍面临重大挑战。现有基于大语言模型的智能家居系统依赖大型模型和计算密集型的推理流水线。这些系统通常部署在基于云的设置中。虽然有效, 但此类设计会引入延迟、需要大量计算资源并引发隐私担忧 [5, 22]。在现实世界的智能家居中, 响应速度和数据隐私是主要关切点, 这使得本地部署的模型极具吸引力。然而, 在本地资源受限的条件下运行需要使用小语言模型。小语言模型推理能力的降低, 给保持可靠性和性能带来了额外挑战。此外, 当前方法对长期个性化的支持有限。尽管大语言模型可以纳入短期对话上下文, 但它们通常缺乏跨交互持续适应用户偏好的机制。因此, 模糊指令会以通用方式被解读, 无法捕捉个人习惯或不断演变的用户行为。在不进行昂贵的模型重训练的情况下支持持续的偏好适应, 仍然是一个悬而未决的挑战。

受这些局限性的启发, 我们提出了**AdaHome**, 一个专为本地部署小语言模型而设计的自适应智能家居助手。AdaHome基于这样一个观察: 并非所有指令都需要相同程度的推理。直接指令可以以最少的处理来执行, 而间接或模糊的输入则受益于额外的推理。为此, AdaHome引入了一个意图感知规划框架, 该框架根据命令类别选择性地应用轻量级推理。这使得系统能在保持动作准确性的同时提高效率。除此之外, AdaHome还融入了一种新颖的偏好适应机制, 以实现持续个性化。它不依赖于提示增强或模型重训练, 而是从已确认和已纠正的交互中构建一个轻量级的偏好记忆。系统利用这个记忆来预测未来的设备选择, 根据语义相似性和时间衰减对过去的交互进行加权。这使得系统能够逐步适应用户行为, 同时对临时偏离保持鲁棒性。

总而言之, 我们的贡献如下:

*本文是以下论文的扩展版本: Eu Jin Lim, Zhaoxing Li 和 Sebastian Stein. 2026. AdaHome: 一种使用本地小语言模型的自适应智能家居助手。载于《国际多模态交互会议 (ICMI '26) 论文集》, 2026年10月05-09日, 意大利那不勒斯。ACM, 美国纽约州纽约市。https://doi.org/10.1145/3776574.3831158

- 我们提出了**AdaHome**，一个自适应智能家居助手，它利用小语言模型实现高效且个性化的控制。
- 我们引入了一个**意图感知规划框架**，它能根据命令类别选择性应用轻量级推理，在保持动作准确性的同时提升效率。
- 我们开发了一种**持续偏好适应机制**，能够从用户交互中实现个性化行为，而无需提示增强或模型重训练。

2 相关工作

2.1 基于大语言模型智能家居助手

近期研究探索了利用大语言模型（LLMs）来改善智能家居环境中的自然语言交互。早期研究表明，大语言模型可以通过解释高级用户意图并生成适当的设备动作来协助智能空间 [2, 6, 16]。基于此方向，更近期的研究将智能家居控制建模为一个推理问题。Sasha系统 [7] 将其表述为一个目标导向推理问题，在执行前将用户意图分解为中间步骤。这种结构化设计提高了动作相关性并减少了误报。然而，它不支持个性化。后续方法将其扩展到基于智能体的设计。例如，SAGE [14] 探索了使用基于LLM的自主智能体进行智能家居控制，从而能更灵活地处理复杂任务。然而，这种灵活性是以更重的推理流程和增加的计算开销为代价的。尽管取得了这些进展，大多数现有系统依赖于像GPT-4这样的大型模型和计算密集的推理过程，这些通常部署在基于云的设置中。这限制了它们在本地、资源受限部署中的实用性。此外，对云服务的依赖引入了隐私风险，例如在数据传输过程中可能发生的数据泄露或未经授权的访问 [17]。

这推动了近期在智能家居环境中使用小语言模型（SLMs）的努力。Harmony [22]引入了一种由本地部署的SLM驱动模块化架构。通过强制执行结构化推理并结合基于规则的控制器进行动作验证，该系统提高了可靠性，并在有限的模型容量下缓解了幻觉问题。类似地，HomeLLaMA [5]采用了一种混合方法，将离线SLM适配与在线交互相结合。这些方法证明了使用SLMs构建本地部署的智能家居助手的可行性。然而，现有系统通常对所有指令统一应用复杂的推理流水线，而无论其复杂程度如何。这可能导致不必要的计算开销，特别是对于那些不需要多步推理的简单指令。先前的研究也报告称，即使对于简单的指令 [23]，此类复杂的推理流水线也可能导致显著的延迟。这凸显了对更高效、自适应机制的需求，该机制能根据用户请求的性质定制推理过程。

表1：用户指令分类。

类别	描述	示例
直接命令	显式设备和动作	打开卧室灯
间接	基于上下文的隐式意图	房间太热
模糊命令	偏好依赖型意图	我想放松一下

2.2 长期个性化

长期个性化是智能家居助手的一项重要功能，因为用户偏好因人而异。尽管近期基于LLM的系统在解释用户意图方面展现出强大的能力，但对用户偏好的持续适应仍未被充分探索。一些先前的工作通过交互历史来融入个性化。例如，SAGE [14] 利用过去的交互，通过检索增强生成（RAG） [27]检索相关的历史上下文来指导未来的决策。类似地，IoTGPT [23] 根据环境属性将用户偏好抽象为结构化表示，并通过基于关键词的匹配将其纳入模型上下文。虽然这些方法实现了一定程度的个性化，但它们依赖于提示层面的增强。随着交互历史的增长，附加的上下文数量也会增加，这使得在小语言模型有限的上下文容量内保持相关性和效率变得困难。更根本的是，此类方法难以平衡稳定性与可塑性之间的权衡，即系统必须保留先前学习到的偏好（稳定性），同时适应新的用户行为（可塑性） [8, 24]。

除此之外，偏好学习也通过模型级适配进行了探索，即对模型进行微调以适应用户偏好 [10]。这些方法计算密集，不适合资源受限的环境，因为更新用户偏好通常需要模型重训练。总体而言，这凸显了在无需依赖提示级增强或模型重训练的情况下，缺乏持续偏好适配的有效机制。

3 系统设计

图1展示了AdaHome的架构，它由一个意图感知规划流水线和一个持续偏好适配模块组成。该流水线根据命令类别将输入路由到直接规划器或推理规划器，而偏好模块则利用语义相似的过往交互来实现个性化响应。

意图分类。我们首先根据其描述的详细程度，将每条用户指令分为三类。如表1所示，指令被分类为直接指令、间接指令或模糊指令，所分配的类别决定了后续的规划策略。直接指令明确指定了目标设备和动作。间接指令隐式地表达了意图，但可以利用当前的设备上下文来解析。相比之下，模糊指令依赖于用户偏好，可能允许存在多个跨用户的有效解释。分类器被实现为一个结构化LLM提示（见附录A），它会输出一个类别标签。其对下游性能的影响在第4.1.2节中进行评估。

AdaHome：一种使用本地小语言模型的自适应智能家居助手

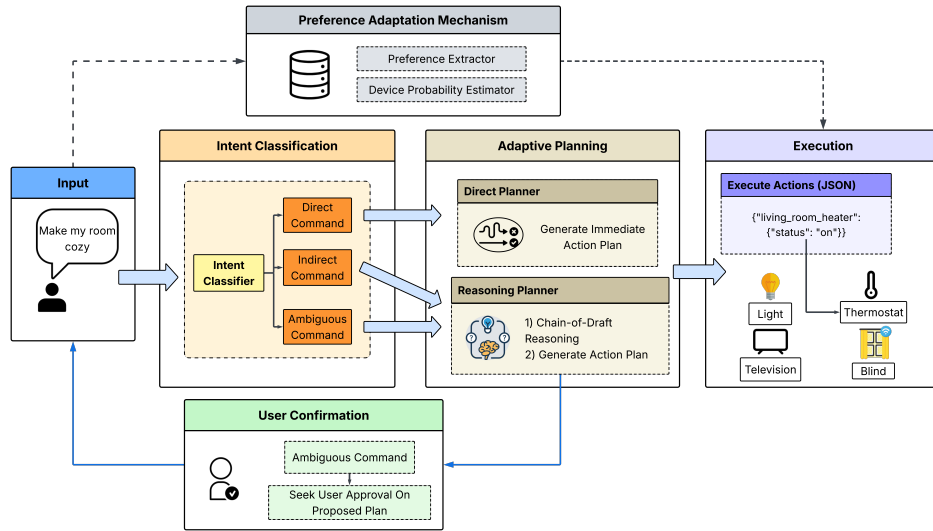


图1：AdaHome系统架构概览，包含一个意图感知规划流水线和一个持续偏好适配模块。该流水线根据命令类别将用户指令路由至直接命令或推理规划器，而偏好模块则利用过去的交互来实现个性化响应。

自适应规划。在意图分类之后，AdaHome将每条指令路由到合适的规划器，仅在必要时应用推理。直接指令由直接规划器处理，它使用无需中间推理的简单提示，从而实现低延迟执行。间接和模糊指令则由推理规划器处理，该规划器在可用设备的上下文中解释用户意图，以生成一个动作计划。为确保可靠性，两个规划器都受到预定义设备模式（附录E）的约束。生成的动作会根据模式中定义的有效设备名称和受支持的属性进行验证。无效的输出会通过约束性重新提示进行纠正，同时保留预期的动作。这种模式引导的过程减少了幻觉，并确保输出的一致性和可执行性。

用户确认步骤。对于模糊指令，AdaHome在执行前引入了用户确认步骤。生成的行动计划会提交给用户批准或修正，以减少因存在多个有效解释而产生的错误，并为门锁解锁或窗帘控制等敏感操作提供额外的安全保障。这同时提高了安全性和可靠性。用户反馈也作为持续偏好适应的监督信号。已确认或修正的操作会被记录在偏好记忆中（详见第3.2节），使系统能随时间推移更好地适应用户特定行为，并减少重复确认的需求。

执行。一旦经过验证和确认，最终的行动计划会以结构化JSON表示，并通过一个执行控制器分派到智能家居环境，该控制器将命令映射到相应的设备API。

为了在资源受限条件下支持高效的决策与个性化，AdaHome将轻量级推理策略与一个用于持续偏好适应的专用机制相结合。

下文我们将描述推理方法（第3.1节）与偏好适应机制（第3.2节）。

3.1 链式草案推理

为处理需要解释的指令，AdaHome在其推理规划器中采用了一种轻量级推理策略。常见的推理方法，如思维链（CoT）[19]和ReAct风格提示[21]，会生成详细的中间推理或涉及多个交互步骤，导致令牌使用量和延迟增加。这使得它们不太适合资源受限的智能家居系统。此外，小语言模型（SLMs）的推理能力有限，即使对于简单指令也可能过度生成不必要的解释，导致输出不稳定[4, 11]。

为了解决这个问题，我们采用链式草案（CoD）提示[20]，它鼓励模型生成一个足以支持决策的最小中间表示（参见附录B）。给定输入后，模型会生成一个紧凑的推理草案来捕获推断出的意图，随后是一个指定设备状态变更的结构化JSON动作。草案和动作均在单次生成中完成。这一设计基于以下观察：智能家居控制主要需要将用户意图映射为可执行动作，而非生成详细解释。因此，冗长的推理会引入额外的延迟，却无法提升决策质量。通过将推理约束在简短的草案中，CoD在保留准确规划所需的基本语义信息的同时，减少了令牌使用量。

此外，小语言模型对冗长的推理输出尤为敏感，这可能导致不一致和幻觉[4, 11]。CoD的紧凑结构提高了输出稳定性和可靠性，使其非常适合端侧部署。图2展示了CoD推理及其输出的一个示例。

草稿提供了推断意图的简洁抽象，而JSON输出则直接指定了可执行的设备动作。

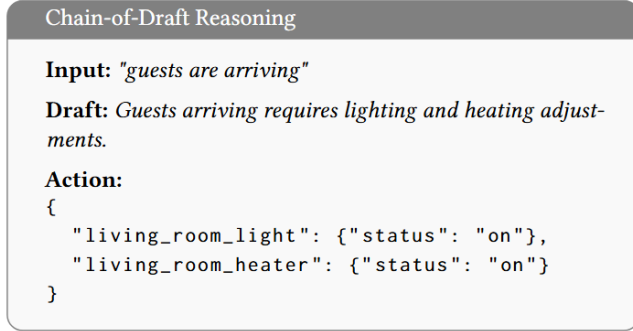


图2：链式草案推理生成JSON动作的一个示例。

3.2 用户偏好适应

用户偏好建模具有挑战性，因为它是高度情境化和动态的。例如，用户发出“让房间变得舒适”的指令时，通常可能偏好明亮的照明，但在某些情况下可能反而偏好昏暗的照明。这种偏离往往是暂时的，用户可能在后续交互中恢复其通常的偏好。一个主要依赖偏好检索（例如，[14, 23]）的系统将难以捕捉这种行为 [24]。这促使我们基于累积的交互历史来建模偏好。

AdaHome 采用了一种轻量级的偏好适应机制，该机制无需模型重训练或提示增强即可运行，使其适合本地部署的小语言模型。每个已确认或已纠正的动作都被视为一个偏好信号。在用户确认后，偏好提取器会从用户指令中提取出一到两个短语的基本意图（参见附录C），过滤掉无关的上下文。这种标准化的表示确保了语义上等价的指令（例如，“电影之夜”和“我们看个电影吧”）映射到相似的嵌入，从而提高了基于相似性检索的可靠性。然后，提取的意图使用句子嵌入模型编码成嵌入向量。该向量被索引到本地向量数据库中，而相应的时间戳和产生的设备状态则存储在本地关系数据库中。在这项工作中，偏好适应是针对二元设备状态 ($x_i \in \{0, 1\}$) 来制定的，其中1表示激活，0表示停用。连续参数（例如，温度）未包含在此公式中，因为对此类属性进行建模需要不同的表示形式。

如图3所示，在推理过程中，系统使用FAISS最近邻搜索在嵌入空间中检索出语义最相似的前 k 个过往交互（我们使用 $k = 10$ ）。随后应用一个余弦相似度阈值来过滤掉弱匹配项，确保仅保留相关的交互用于预测。如果没有交互满足此相似度阈值，系统则按第3节所述，继续执行意图感知规划流水线。给定一个新指令，提取出的意图被

嵌入为 v_{new} 。每个检索到的记忆 i 包含一个历史嵌入 v_i 、一个二进制设备状态 x_i 和一个时间戳 t_{i0} 。

为了估计每次先前交互对当前决策应产生多大的影响，我们计算一个语义时间衰减权重 w_i ：

$$w_i = S(v_{\text{new}}, v_i)^\gamma \cdot e^{-\lambda \Delta t_i} \quad (1)$$

其中 $S(\cdot, \cdot)$ 表示余弦相似度， Δt_i （以天为单位）是自交互 i 被记录以来经过的时间，而 λ 控制着较早交互失去影响力的速率。相似度项捕捉了当前意图与过去意图之间的语义相关性，而 γ 控制着权重的锐度，增加了高度相似与弱相似交互之间的对比度。在我们的实现中，我们固定 $\gamma = 3$ ，这在不将权重过度集中于单一交互的情况下提高了选择性。指数项优先考虑更近期的交互，这与记忆建模中的时间遗忘模式 [26] 一致。这里我们设定 $\lambda = 0.1$ ，这使得过去交互的影响力在大约 7 天后降至约 0.5。这个时间尺度与在智能家居使用 [15] 中观察到的每周重复模式相符。因此，较旧或不频繁的交互会继续以递减的权重做出贡献。

随后，使用核回归估计器 [12, 18] 计算设备 d 应被激活的概率：

$$P(d = 1) = \frac{\sum_i w_i x_i}{\sum_i w_i} \quad (2)$$

其中权重作为嵌入空间上的核函数。它基于类似过往交互的加权平均来估计激活设备的可能性。在语义上相似且发生时间更近的交互，对最终决策的贡献更强。随着时间的推移，频繁的行为会累积并主导预测，使系统对偶尔的偏差具有鲁棒性。同时，如果用户持续改变其行为，新累积的偏好将逐渐移动决策边界。

最后，通过确认或纠正来整合用户反馈，并将其作为一个新的偏好元组 ($v_{\text{new}}, x_{\text{new}}, t_{\text{new}}$) 存储并添加到记忆中。这形成了一个持续反馈循环，使 AdaHome 能够随时间推移优化其行为，而无需重新训练。

4 评估

我们进行了一项定量评估，从两个方面评估AdaHome。首先，我们检验系统将多样化的用户指令转化为结构化设备动作的能力。其次，我们评估其支持随时间持续适应用户偏好的能力。我们的评估采用统一小模型设置，以反映现实的边缘部署约束。所有系统均使用相同的轻量级模型 (Llama 3.2-3B [3]) 实现。在所有实验中，Ollama¹ 被用作推理引擎，并且模型使用贪婪解码（温度 = 0.0）执行，以最小化随机性。实验在一个由固定的智能家居设备模式定义的受控环境中进行，该模式指定了可用设备、当前状态和允许的动作。

¹<https://ollama.com>

AdaHome: 一种使用本地小语言模型的自适应智能家居助手

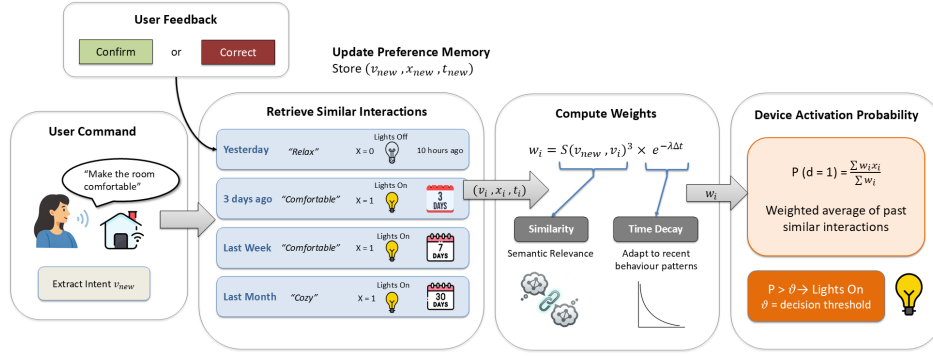


图3: AdaHome偏好适应机制概览。系统检索语义相似的过往交互，计算权重，并估算设备激活概率。当可用时，用户反馈会被纳入以更新偏好记忆。

这确保了跨系统生成输出的一致性验证。为了反映资源受限的边缘部署，所有系统都在一台配备AMD Ryzen 5 220 CPU和16GB RAM的机器上本地执行，该配置代表了一款低成本消费设备（约200-300英镑）。

4.1 对比研究

我们在单轮交互设置中比较所有系统，其中对先前交互历史和用户反馈的记忆访问均被禁用。这隔离了每个系统的核心决策行为，并移除了任何外部因素。每个实验重复三次以确保一致性。

4.1.1 实验设置。

智能家居设置。 我们建立了一个标准化的智能家居环境，包含12个预定义设备，涵盖照明、气候控制、电器和安防等多个功能类别。每个设备都与一组固定的可控属性相关联。详细的设备模式见附录E。

基线模型。 我们将AdaHome与三种具有代表性的基于LLM的智能家居基线模型进行比较。我们专注于在统一小模型设置下评估它们的架构行为。此设置反映了实际的边缘部署场景，并能在有限的模型容量下对不同系统设计的工作方式进行受控分析。每个基线模型均被复现，以保留其核心架构设计，同时在相同的模型和环境约束下运行。

Sasha [7] 是一个面向目标的LLM智能家居助手，它将指令处理分解为可行性检查、设备筛选和动作规划。我们按照原论文详述的多阶段流程复现了Sasha，并禁用了其交互式反馈循环，以保持与我们自动化评估的一致性。

SAGE [14] 是一个基于代理的系统，执行基于工具的迭代推理。我们在固定的设备模式内重新实现了其协调器和工具选择循环，专注于设备控制任务，并排除了辅助功能（例如，网络搜索和视觉输入）。

Harmony [22] 是一个为本地部署的轻量级模型设计的模块化智能家居助手。我们复现了其消息处理、基于事实的推理和基于规则的验证三阶段流程，并禁用了记忆组件。

数据集。 我们使用源自Sasha数据集（40条指令）和SAGE数据集（50条指令）的指令构建了一个评估数据集，仅保留那些与我们预定义设备模式兼容的指令。我们进一步通过手动构建的指令扩展了该数据集，以确保对表1中三个类别的均衡覆盖。最终数据集包含90条指令：30条直接命令、30条间接命令和30条模糊命令。完整数据集在附录F中提供。

评估指标。 我们从两个维度评估所有系统：行动准确率和计算效率。对于直接指令，其期望结果是明确指定的，我们使用基于最终设备状态精确匹配的二元成功率。为了进一步描述错误类型，我们还额外报告误报率 (*FP*) 和漏报率 (*FN*)，其中 *FP* 表示存在任何额外被激活的设备，而 *FN* 表示未能激活所需的设备。

对于间接指令和模糊指令，严格的精确匹配限制性过强，因为期望结果并未完全指定。间接指令通常暗示一个主要可操作设备（例如，“客厅有灰尘”暗示启动扫地机器人），但像“打开空气净化器”这样的额外行动也是合理的，不应被惩罚。模糊指令本质上依赖于偏好，因此存在多个有效的行动计划。例如，“晨间惯例”可能对应不同用户的不同设备组合。在这两种情况下，我们基于语义恰当性而非精确匹配来评估输出。具体而言，我们采用LLM即裁判 [25] 来评估生成的动作结果在给定指令和设备状态下是否恰当。为了减少评估偏差，裁判使用来自不同系列、规模更大的模型（Qwen2.5-14B [13]）和一个固定的评估提示（见附录D.1），并输出一个二元成功/失败判定。示例输出见附录D.2。

为评估基于LLM的评估的可靠性，我们向评判器查询三次并采用多数投票。我们进一步将

其输出与人工标注进行对比，该标注基于涵盖所有系统和指令类别的48个分层样本子集。人工标注由研究团队成员独立完成，遵循提供给LLM评判器的相同评估标准。LLM评判器与人工评估达成高度一致（Cohen's $\kappa = 0.834$ ），表明在我们的设定中，它为人工作评估提供了可靠的替代方案。

为了评估计算效率，我们测量端到端推理延迟，并报告每次交互的输入和输出词元数量。

4.1.2 结果。 汇总结果呈现在表2中，图4展示了总结性可视化图表。

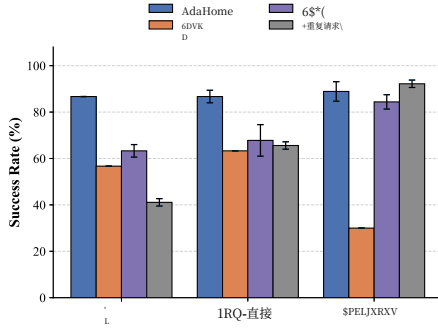


图4：直接指令、间接指令和模糊指令类型的成功率。误差线表示三次运行的标准差。若误差线不可见，则方差可忽略不计。

动作准确性。 AdaHome 在直接指令上取得了最高的成功率（86.7%），优于所有基线模型。Harmony 表现出较高的误报率（37.8%），因为其强制性的推理阶段倾向于过度解读显式指令，导致产生虚假的设备激活。相反，Sasha 则受困于较高的漏报率，因为相关设备在中间可行性检查和过滤阶段经常被排除在外。同样，SAGE 依赖于基于工具的迭代推理，这引入了对多步决策和终止控制的依赖。在小模型约束下，系统可能反复调用中间工具（例如，能力检索），而无法推进到最终的动作执行。这些结果揭示了将复杂推理流水线应用于小语言模型的一个关键局限。当任务本身已明确定义时，额外的推理会增加产生幻觉或意图偏离的可能性。AdaHome 通过使用一个直接的、专注于指令的提示词，并结合模式引导的生成过程来缓解此问题，从而在有限的模型容量下提高了执行可靠性。

在间接指令方面也观察到了类似的模式。AdaHome 保持着最高的成功率（86.7%），而 Sasha、SAGE 和 Harmony 则取得了大约 60-70% 的相当性能。在此类别中，AdaHome 采用了一种轻量级的草稿链推理策略来解析隐含意图。与繁重的推理流水线不同，CoD 提供了一种简洁的中间表示，用以指导动作生成，而无需引入不必要的推理步骤。这使得模型能够

捕捉隐含意图，同时避免了在更复杂的推理框架中观察到的过度生成和错误传播。结果表明，在 SLM 约束下，结构化、最小化的推理足以解决间接指令。

对于模糊指令，则出现了不同的趋势。Harmony取得了最高的成功率（92.2%），其次是AdaHome（88.9%）和SAGE（84.4%）。这反映了更具表现力的推理流水线在探索未充分指定的输入可能解释的更大空间方面的优势。相比之下，AdaHome通过其轻量级的CoD策略优先考虑效率，这可能限制了穷尽式探索，但仍实现了有竞争力的性能。值得注意的是，Sasha表现不佳（30.0%），这主要归因于其保守设计，即在无法自信解决任务时会选择放弃。

意图分类的影响。 由于AdaHome的性能依赖于将指令动态路由到正确的规划器，我们进一步研究了意图分类错误的影响。意图分类器在评估数据集上的准确率为75.6%。直接指令偶尔会被误分类为间接指令，这导致一些直接输入被路由到推理规划器。虽然这会引入额外的延迟，但通常能保持正确的动作生成。对于间接和模糊指令，路由错误主要影响是否触发确认步骤，因为这两种情况都由同一个推理规划器处理。总体而言，误分类影响的是执行效率而非动作正确性，这表明系统对路由错误具有鲁棒性。

计算效率。 AdaHome在所有系统和命令类别中实现了最低的端到端延迟和最小的令牌占用。这种效率源于其意图感知路由策略，该策略仅在需要时才调用推理。Harmony由于冗长的推理和额外的基于规则的验证而产生了最高的延迟和令牌使用量，而SAGE则因其迭代的基于工具的推理循环而变慢。总体而言，结果揭示了推理复杂度和计算效率之间的明显权衡。尽管更具表现力的推理流水线可以在模糊场景中提升性能，但它们会带来巨大的开销，并且在小模型约束下鲁棒性较差。AdaHome证明，一种轻量级且有选择性地应用的推理策略可以在准确率和效率之间实现更有利的平衡。图5进一步说明了这种权衡。

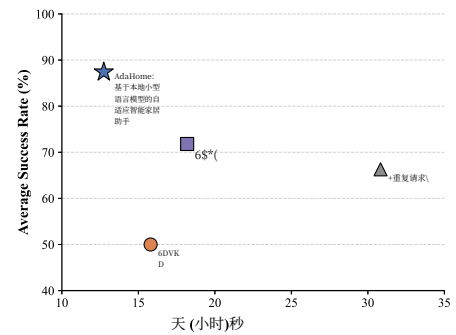


图5：所有指令类别中平均端到端延迟与平均成功率之间的整体权衡。

表2：三次运行平均的对比研究结果。对于间接命令和模糊指令，误报率和漏报率不适用（-），因为成功率通过目标达成度确定。每个类别中的最佳结果已加粗。

类别	系统	成功率 (%)	误报率 (%)	漏报率 (%)	延迟 (秒)	输入令牌数	输出令牌
直接命令	Sasha	56.7	0.0	43.3	15.07	619	58
	SAGE	63.3	2.2	36.7	17.95	757	105
	Harmony	41.1	37.8	31.1	28.84	799	168
	AdaHome	86.7	0.0	13.3	10.48	441	32
间接	Sasha	63.3	-	-	17.47	659	67
	SAGE	67.8	-	-	17.45	660	94
	Harmony	65.6	-	-	30.71	812	171
	AdaHome	86.7	-	-	13.28	470	50
模糊命令	Sasha	30.0	-	-	14.87	463	74
	SAGE	84.4	-	-	19.16	660	114
	Harmony	92.2	-	-	32.93	819	195
	AdaHome	88.9	-	-	14.47	463	65

4.2 偏好适应分析

我们在多轮交互设置中评估AdaHome，以评估其在重复交互中支持持续用户偏好适应的能力。与单轮评估不同，本实验捕捉了跨交互过程中不断演变的用户行为。目标是衡量系统是否能保持一致的偏好，从临时偏离中恢复，并适应偏好的变化。

基线模型。我们将AdaHome与一个检索增强生成（RAG）基线进行比较，该基线作为一种代表性的提示增强方法 [9, 14]。在此设置中，过去的交互记录被存储，并根据与当前用户指令的语义相似性进行检索。在推理过程中，检索到的偏好会被整合到模型上下文中。当用户偏好发生变化时，存储的记忆会相应更新。为确保公平比较，RAG基线使用相同的模型提示，并应用了链式草案推理。

数据集。我们使用对比研究中的模糊指令构建了一个纵向数据集，以模拟用户随时间的重复交互（参见附录G）。该数据集包含30个序列，每个序列有8次交互轮次。我们设计每个序列遵循三种行为模式之一。在稳定例程中，底层偏好在所有轮次中保持一致（例如，偏好A → A → A）。在临时偏离中，会发生短期变化，但随后恢复为原始行为（例如，A → A → B → A）。在偏好转变中，用户永久性地转向新的偏好（例如，A → A → B → B）。这些序列的示例见附录G.2。

评估指标。持续的偏好适应需要在稳定性与可塑性之间取得平衡，系统必须保留先前习得的偏好，同时对新用户行为 [8]保持响应。遵循这一原则，我们使用四个互补的指标来评估偏好适应。偏好一致性 在稳定序列上评估，衡量系统在语义相似的指令下是否重现相同的偏好。恢复率 在偏差序列上评估，衡量系统是否能在临时扰动后正确

恢复到原始偏好。这评估了对噪声的鲁棒性。适应成功率 和 适应延迟 在偏好转变序列上评估。它们衡量系统是否以及多快能与新偏好保持一致。

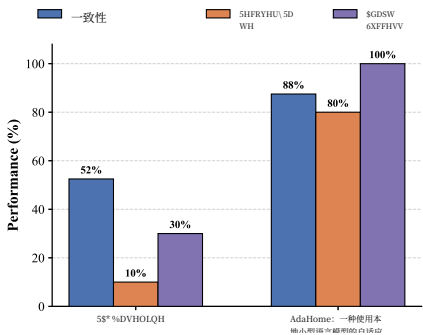


图6：多轮次偏好适应性能对比：AdaHome 与 RAG

4.2.1 结果。结果如图6所示。RAG基线实现了中等稳定的准确率（52.5%），表明其在重复交互中能够部分保持一致的偏好。然而，其恢复率非常低（10.0%），这表明它对最近的交互高度敏感，并且在遇到临时偏离时，倾向于覆盖先前习得的偏好。尽管RAG适应速度相对较快（平均2.0轮次收敛至新偏好），但其较低的适应成功率（30.0%）表明，这些更新常常不一致，且不能可靠地反映真实的偏好转变。

相比之下，AdaHome实现了较高的稳定准确率（87.5%）、强大的恢复能力（80.0%）以及100.0%的适应成功率。虽然适应速度稍慢（平均需要2.6轮次才能收敛），但更新更为可靠。这表明系统能够保留先前学习到的偏好，并有效地

随时间适应新的偏好。与RAG相比，AdaHome在稳定性和可塑性之间提供了更平衡的权衡。

4.3 消融研究

我们进行了一项消融研究，以检验AdaHome偏好适应机制中关键组件的贡献。我们通过完全移除记忆来评估该机制的作用，并进一步分离机制内偏好提取的效果。所有实验均使用与偏好适应分析中描述的相同数据集和评估指标。我们比较了三种配置：

- **无记忆**：每个回合独立生成动作，无法访问先前的交互。
- **AdaHome（无提取器）**：偏好机制直接应用于原始用户指令，不使用偏好提取器。
- **AdaHome（带提取器）**：包含偏好提取器的完整系统。

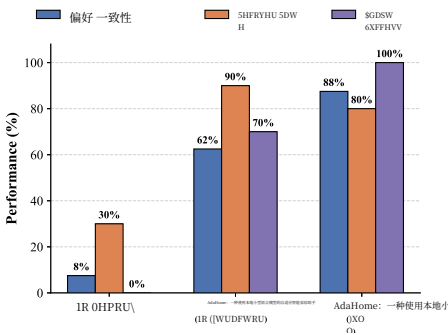


图7：AdaHome组件消融研究：记忆与偏好提取的效果。

4.3.1 结果。图7展示了所有配置的性能。无记忆基线在所有指标上表现均不佳，仅达到7.5%的稳定准确率，且无法适应偏好转变。这证实了不利用交互历史就无法实现偏好适应。

不带偏好提取的AdaHome实现了最高的恢复率（90.0%）和更高的适应成功率（70.0%），反映出对近期用户交互的强响应性。直接在原始用户指令上操作使得系统对变化高度敏感。例如，考虑一个临时偏离序列（电影之夜 → 我们看电影 → 影院模式 → 电影之夜）。没有偏好提取，系统会独立处理每条指令，使其能够根据最近的输入快速切换行为，从而在用户恢复原始偏好时实现快速恢复。然而，这种反应性是以较慢的适应速度为代价的（平均需要3.1轮次才能收敛）。自然语言表达的变化会稀释语义相似性，需要更多观察才能可靠地区分临时偏离与真实的偏好转变。

引入偏好提取既提高了适应可靠性，也提升了效率。通过提取结构化的偏好表示，提取器能在

嵌入空间中对语义等价的输入进行对齐，从而实现更一致的相似度匹配。与没有提取功能的变体相比，完整系统实现了更高的稳定准确率和适应成功率，同时减少了适应延迟。这表明偏好提取减少了语义噪声，并实现了更准确、更一致的偏好更新。

5 局限性与未来工作

我们的评估侧重于重复交互中的多轮偏好适应。虽然这证明了系统适应不断变化的用户行为的能力，但仍需进行长期用户研究，以评估其在持续使用下的鲁棒性。此外，当前的偏好适应机制仅对二元设备状态进行建模，将其扩展以处理连续设备属性是未来工作的重要方向。AdaHome 还通过回退到大语言模型推理来处理冷启动场景。虽然这确保了系统从一开始就具备功能性行为，但减少对早期用户反馈的依赖并加速个性化，仍然是待解决的挑战。最后，我们当前的实验侧重于基于文本的交互。整合额外的多模态信号，如语音或手势，将是迈向更灵活、更自然的智能家居交互的重要下一步。

6 结论

我们提出了 AdaHome，一个用于本地部署的小语言模型的自适应智能家居助手。通过根据指令类型选择性应用轻量级推理，AdaHome 在小模型约束下，比现有方法实现了更强的效率-准确率权衡。我们的评估进一步表明，与 RAG基线 相比，AdaHome 支持更可靠的多轮偏好适应，且无需提示增强或模型重训练。这些发现凸显了小语言模型在实现高效、个性化的智能家居交互方面的潜力。

7 安全与负责任创新

AdaHome 旨在实现保护隐私、安全可靠的智能家居交互。该系统完全在本地运行，使用小语言模型，避免将敏感用户数据传输到外部服务器。潜在风险包括因误解用户意图而导致的设备错误激活，以及对可能无法反映真实偏好的短期行为的过度适应。为降低这些风险，AdaHome 采用模式约束验证来强制执行有效的设备状态，并为模糊指令设置了用户确认步骤。此外，系统利用持续用户反馈来优化偏好表征，并随时间推移提高鲁棒性。

参考文献

[1] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, 以及 Hervé Jégou. 2024. Faiss库. (2024 2401 08281 []). arXiv: .cs.LG Simone Gallo, Fabio Paternò, 以及 Alessio Malizia. 2024. 一种利用大语言模型创建自动化的对话代理。个人与普适计算28, 6 (2024), 931–946. [3] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, 等人. 2024. Llama 3模型群. arXiv预印本arXiv:2407.21783(2024)。

AdaHome: 一种使用本地小语言模型的自适应智能家居助手

- [4] Dongge Han, Menglin Xia, Daniel Madrigal Diaz, Samuel Kessler, Ankur Mallick, Xuchao Zhang, Mirian Del Carmen Hipolito Garcia, Jin Xu, Victor Rühle, 以及 Saravan Rajmohan. 2025. 通过蓝图和提示模板搜索增强小语言模型的推理能力. arXiv预印本arXiv:2506.08669 (2025). [5] Xinyu Huang, Leming Shen, Zijing Ma, 以及 Yuanqing Zheng. 2026. 通过定制化小语言模型实现隐私保护与个性化的智能家居. IEEE移动计算汇刊 (2026). [6] Evan King, Haoxiang Yu, Sangsu Lee, 以及 Christine Julien. 2023. "准备开派对吧": 借助大语言模型探索更智能的智能空间. arXiv预印本arXiv:2303.14143 (2023). [7] Evan King, Haoxiang Yu, Sangsu Lee, 以及 Christine Julien. 2024. Sasha: 智能家居中基于大语言模型的创造性目标导向推理. ACM交互式、移动、可穿戴与普适技术汇刊 8, 1 (2024), 1–38. [8] Song Lai, Zhe Zhao, Fei Zhu, Xi Lin, Qingfu Zhang, 以及 Gaofeng Meng. 2025. 帕累托持续学习: 面向动态稳定性-可塑性权衡的偏好条件学习与适应. 载于 AAAI人工智能会议论文集, 第39卷. 18008–18016. [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, 等人. 2020. 面向知识密集型NLP任务的检索增强生成. 神经信息处理系统进展 33 (2020), 9459–9474. [10] Shuozhe Li, Daniel Yamakov, 以及 Christine Julien. [未注明日期]. 重新构想智能家居: 人机协作的大语言模型实现高效安全的家庭自动化. ([未注明日期]). [11] Yuetai Li, Xiang Yue, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, Bhaskar Ramasubramanian, 以及 Radha Poovendran. 2025. 小模型难以从强推理者处学习. 载于计算语言学协会发现: ACL 2025. 25366–25394. [12] Elizbar A Nadaraya. 1964. 论回归估计. 概率论及其应用 9, 1 (1964), 141–142. [13] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, 以及 Zihan Qiu. 2025. Qwen2.5技术报告. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115> [14] Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, Xue Liu, 以及 Gregory Dudek. 2024. 基于自主大语言模型代理的AIoT智能家居. IEEE物联网期刊 12, 3 (2024), 2458–2472. [15] Abdussalam Salama, Reza Saatchi, Maryam Bagheri, Mahpara Saleem, 以及 Muhammad Usman Shad. 2025. 集成于智能手机的长短期记忆实时家庭监测应用的开发与评估. 算法 18, 12 (2025), 780. [16] Yingtian Shi, Xiaoyi Liu, Chun Yu, Tianao Yang, Cheng Gao, Chen Liang, 以及 Yuanchun Shi. 2024. 弥合智能家居中自然用户表达与复杂自动化编程之间的鸿沟. arXiv预印本arXiv:2408.12687 (2024). [17] Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjin Wang, Rui Li, Junjie Xu, Xianfeng Tang, 等人. 2025. 大语言模型时代下的小语言模型综述: 技术、增强、应用、与大语言模型的协作及可信度. ACM智能系统与技术汇刊 16, 6 (2025), 1–87. [18] Geoffrey S Watson. 1964. 平滑回归分析. Sankhyā: 印度统计学杂志, A辑 (1964), 359–372. [19] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, 等人. 2022. 思维链提示激发大语言模型的推理能力. 神经信息处理系统进展 35 (2022), 24824–24837. [20] Silei Xu, Wenhao Xie, Lingxiao Zhao, 以及 Pengcheng He. 2025. 草稿链: 通过少写实现更快思考. arXiv预印本arXiv:2502.18600 (2025). [21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, 以及 Yuan Cao. 2022. React: 在语言模型中协同推理与行动. 载于第十一届国际学习表征会议. [22] Ziqi Yin, Mingxin Zhang, 以及 Daisuke Kawahara. 2025. Harmony: 一个具备人类感知、响应能力、模块化且本地部署大语言模型的助手. 载于2025年ACM普适与泛在计算国际联合会会议配套论文集. 126–130. [23] Chaerin Yu, Chihun Choi, Sunjae Lee, Hyosu Kim, Steven Y Ko, Young-Bae Ko, 以及 Sangeun Oh. 2026. 利用大语言模型实现高效个性化的智能家居自动化. arXiv预印本arXiv:2601.04680 (2026). [24] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, 以及 Ji-Rong Wen. 2025. 基于大语言模型的智能体记忆机制综述. ACM信息系统汇刊 43, 6 (2025), 1–47. [25] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, 等人. 2023. 使用MT-Bench和聊天机器人竞技场评判LLM即法官. 神经信息处理系统进展 36 (2023), 46595–46623. [26] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, 和 Yanlin Wang. 2024. Memo-rybank: 利用长期记忆增强大语言模型. 载于AAAI人工智能会议论文集, 第38卷. 19724–19731. [27] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, 和 Ji-Rong Wen. 2025. 用于信息检索的大语言模型: 综述. ACM信息系统汇刊44, 1 (2025), 1–54.

意图分类提示

意图分类提示

你是智能家居助手的意图分类器。

输入：

- 用户指令：{ user_command }

将用户指令精确分类到以下一个类别中：- 直接命令：明确的设备控制指令，包含清晰指定的动作和目标设备（例如，“打开客厅灯”）- 间接：隐含但可从上下文推断的意图（例如，“太热了”）- 模糊命令：主观的、偏好依赖的或类似日常习惯的请求，可能根据用户有多个有效解释（例如，“我想放松一下”）

请仅以有效的JSON格式回应。"category"键的值必须严格为以下之一：DIRECT、INDIRECT 或 AMBIGUOUS。

示例：

```
{"category": "DIRECT"}
```

B 链式草稿提示

推理规划器提示

你是一个智能家居助手的推理规划器。

输入：

- 设备状态：{ devices } - 用户指令：{ user_command }

根据用户指令推断必要的设备状态变更。逐步思考，但只保留一份最多8个词的最简推理草案。

请严格按照以下格式回复：草案： <简洁的推理草案 >操作：

<包含状态变更的有效JSON对象 >

仅使用设备状态中提供的设备名称。

C 偏好提取提示

偏好提取提示

你是一个智能家居助手的偏好提取器。输入：- 用户指令：{ user_command } 提取一个简洁的1-2词核心意图，以捕捉用户潜在目标。示例：- "我想放松一下" -> {" 核心_意图 ": " relax "} 请仅使用此格式回复有效的JSON：{" 核心_意图 ": "..."}

D LLM作为裁判的评估

D.1 提示

LLM即裁判评估提示

你是一个智能家居系统评估员。

输入：

- 设备状态：{ devices } - 用户指令：{ user_command } - 代理动作：{ output }

评估代理动作是否成功满足了用户的目标。

评估标准：

- 成功率 (1)：操作在逻辑上和安全地满足了用户在初始状态下的意图。允许多种有效解决方案。 - 失败 (0)：操作未能满足意图或激活了无关设备。

指令：

- 提供简洁的推理（1-2句话）。 - 然后输出最终的二元评分。

请仅使用此格式返回有效的JSON：{"推理": "...", "成功率": 1}

D.2 示例输出

成功示例

指令: timeforbed智能体动作: {"living_room_light":{"状态": "off"},"bedroom_light":{"状态": "off"}}

评判输出:
{"reasoning": "关灯为睡眠创造了合适的环境, 并符合用户的意图。", "success":1}

失败示例

指令: 电影之夜智能体动作: {"living_room_light":{"状态": "on"}}

评判输出:
{"reasoning": "仅打开灯并不能营造出电影之夜通常预期的环境, 例如打开电视。", "success": 0}

E 智能家居设置

表3: 智能家居设备模式

设备	Type	属性
客厅_室_灯	照明	状态, 亮度
卧室_灯	照明	状态, 亮度
客厅_室_加热器	气候控制	状态, 温度
卧室加热器_	气候控制	状态, 温度
客厅_室_空调	气候控制	状态
卧室 _ac	气候控制	状态
客厅_电视_	电器	状态
咖啡机_	电器	状态
空气_净化器	电器	状态
卧室百叶窗_	固定装置	状态 (开启/关闭)
扫地机器人_	电器	状态 (清洁中/已归位)
前门锁_ _	安防	状态 (已上锁/未上锁)

F 评估数据集

评估数据集分为三类：直接指令、间接指令和模糊指令。直接指令通过与预定义目标状态进行精确匹配来评估，而间接指令和模糊指令由于存在多种有效解释，则使用LLM即法官进行评估。

F.1 直接指令

表4：直接指令及其预期目标状态。

用户指令	预期目标状态
打开客厅电视	{ "living_room_television": { "状态": "on" } }
打扫卧室	{ "robot_吸尘器": { "状态": "cleaning" } }
锁好房子	{ "front_door_lock": { "status": "locked" } }
煮一些咖啡	{ "咖啡_机": { "状态": "on" } }
关闭卧室灯	{ "卧室_灯": { "状态": "off" } }
打开空气净化器	{ "空气_净化器": { "状态": "on" } }
关闭卧室百叶窗	{ "卧室_百叶窗": { "状态": "已关闭" } }
解锁前门	{ "前_门锁": { "状态": "已解锁" } }
打开客厅空调	{ "客厅_空调": { "状态": "on" } }
关闭客厅电视	{ "living_room_television": { "状态": "off" } }
关闭客厅取暖器	{ "living_room_heater": { "状态": "off" } }
打开卧室百叶窗	{ "卧室_百叶窗": { "状态": "open" } }
打开卧室空调	{ "卧室_空调": { "状态": "on" } }
让扫地机器人返回充电	{ "robot_吸尘器": { "状态": "docked" } }
打开卧室加热器	{ "卧室_暖气": { "状态": "on" } }
停止咖啡机	{ "咖啡_机": { "状态": "off" } }
调暗客厅灯	{ "living_room_light": { "状态": "on", "brightness": "low" } }
锁上前门	{ "front_door_lock": { "status": "已锁定" } }
关闭客厅空调	{ "客厅_空调": { "状态": "关闭" } }
将卧室灯设置为高亮度	{ "卧室_灯": { "状态": "on", "亮度": "高" } }
启动扫地机器人	{ "扫地_机器人": { "状态": "清洁中" } }
打开卧室灯	{ "卧室_灯": { "状态": "on" } }
打开咖啡机	{ "咖啡_机": { "状态": "on" } }
关闭空气净化器	{ "空气_净化器": { "状态": "关闭" } }
现在关闭卧室百叶窗	{ "卧室_百叶窗": { "状态": "已关闭" } }
将客厅取暖器设置为高档	{ "living_room_heater": { "状态": "on", "temperature": "high" } }
打开客厅灯	{ "living_room_light": { "状态": "on" } }
关闭客厅灯	{ "living_room_light": { "状态": "off" } }
打开卧室的百叶窗	{ "卧室_百叶窗": { "状态": "open" } }
关闭卧室空调	{ "卧室_空调": { "状态": "关闭" } }

F.2 间接指令

表5：间接指令。

用户指令	用户指令
让这里别那么冷	让一些自然光照进卧室
又冷又黑	我忘记锁好房子了
太热了	客厅里太暗了，没法看书
卧室里太亮了	我想看一部电影
我在客厅里什么都看不见	卧室里的空气不新鲜了
空气感觉很闷	我需要喝点咖啡来提神
卧室里太热了	我的脚在客厅里很冷
太阳在卧室里刺得我眼睛疼	晨光把我弄醒了
太吵了，把那个关掉	地板上全是碎屑
清理地板上的这团乱糟糟的东西	确保房子在夜晚是安全的
我需要咖啡因提神	起居区热得像蒸笼一样
确保没人能进来	我要睡觉了，关掉卧室灯
客厅里有点变热了	这么多灰尘，我呼吸不畅
你能净化一下这里的空气吗	我把薯片撒得到处都是
我在发抖	我在卧室里冻坏了

F.3 模糊指令

表6：模糊指令。

用户指令	用户指令
营造舒适氛围	我想放松一下
派对准备	锻炼时段
电影之夜	我想读本书
早安	准备上班
睡觉时间	约会之夜
我要去上班了	起床时间
欢迎回家	我今天感觉不舒服
学习时间	让我们度过一个慵懒的周日
浪漫晚餐	专注模式
我偏头痛	孩子们睡着了
让我们做些瑜伽	我要举办晚宴
游戏时间	节能模式
房屋清洁日	我需要集中注意力
客人即将抵达	锻炼后放松
午睡	家庭桌游之夜

G 纵向偏好数据集

该数据集源自评估数据集中的模糊指令类别，用于模拟纵向用户交互。它通过一组预定义的模糊场景构建，每个场景关联多个释义和两种备选的偏好配置（偏好A和偏好B）。对于每个场景，我们通过采样释义并根据三种行为模式（稳定例程、临时偏离和偏好转变）分配偏好轨迹来生成交互序列。表7总结了用于构建数据集的模糊场景、释义及相应的偏好配置。

G.1 偏好场景

表7：用于生成纵向交互序列的模糊场景、相关释义及偏好配置。LR表示客厅，BR表示卧室。

意图	释义	偏好A	偏好B
Cozy	让客厅舒适 让我的房间舒适 让它舒适 舒适夜晚模式	客厅暖气打开	客厅暖气打开 + 客厅灯光关闭
离家模式	我要离开房子了 现在要出门了 再见，房子 我要出门了	锁前门 +客厅灯关闭 +卧室灯关闭	锁前门 + 吸尘器开启 +空气净化器开启
电影之夜	电影之夜 我们看部电影吧 影院模式 看电影时间	客厅电视开启 +客厅灯开启	客厅电视开启 + 客厅灯关闭
专注模式	我需要进入专注模式 学习时间 帮我集中注意力 我需要安静地完成工作	客厅灯开启 + 客厅电视关闭	客厅灯开启 + 空气净化器开启
Nap	小憩一下 我需要睡一会儿 我想休息一下 打算小睡一会儿	卧室窗帘关闭 +卧室灯关闭	卧室灯关闭

G.2 示例交互序列

示例：稳定例程（电影之夜）

Pattern: A → A → A → A → A → A → A → A

意图：电影之夜

轮次：

- T1: 电影之夜 → 电视开启 +灯光开启
- T2: 我们看部电影吧 →电视开启 + 灯光开启
- T3: 电影之夜 → 电视开启 +灯光开启
- T4: 电影之夜 → 电视开启 +灯光开启
- T5: 电影之夜 → 电视开启 +灯光开启
- T6: 电影之夜 → 电视开启 +灯光开启
- T7: 电影之夜 → 电视开启 +灯光开启
- T8: → 电视开启 +灯光开启

示例：临时偏差（舒适）

Pattern: A → A → A → A → A → B → A → A

意图：舒适

轮次：

- T1: 舒适夜晚模式 →打开暖气
- T2: 让我的房间舒适 →打开暖气
- T3: 让它舒适 →打开暖气
- T4: 舒适夜晚模式 →打开暖气
- T5: 让我的房间舒适 →打开暖气
- T6: 舒适夜晚模式（我头疼） → 打开暖气 + 灯光关闭
- T7: 让我的房间舒适 →打开暖气
- T8: 让我的房间舒适 →打开暖气

示例：偏好转变（电影之夜）

Pattern: A → A → A → A → B → B → B → B

意图：电影之夜

轮次：

- T1: 看电影时间 → 电视开启 +灯光开启
- T2: 看电影时间 → 电视开启 +灯光开启
- T3: 影院模式 → 电视开启 + 灯光开启
- T4: 看电影时间 → 电视开启 +灯光开启
- T5: 影院模式 → 电视开启 + 灯光关闭
- T6: 看电影时间 → 电视开启 +灯光关闭
- T7: 影院模式 → 电视开启 + 灯光关闭
- T8: 我们来看部电影吧 →电视开启 + 灯光关闭